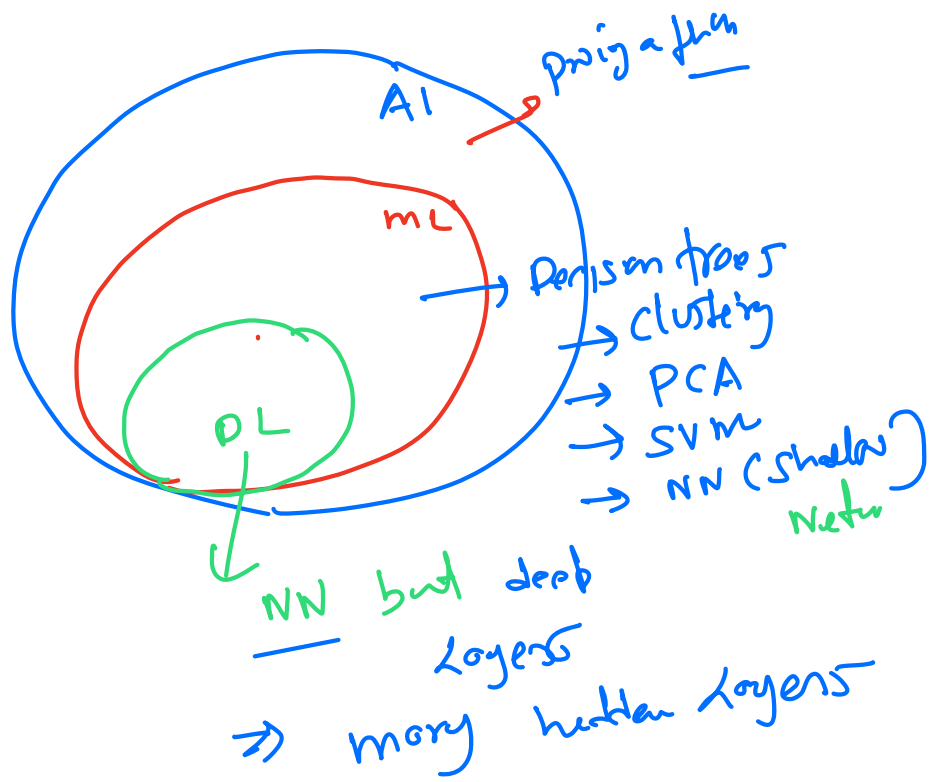

Improving Deep Neural Networks

Optimization · Regularization · Tuning · Transfer Learning

Dr. Rishikesh Yadav

Assistant Professor, IIT Mandi

June 4, 2026



Course Outline

Module 1: Optimization

- ✓ Gradient Descent Variants
- ✓ Momentum & RMSProp
- ✓ Adam Optimizer
- ✓ Learning Rate Scheduling

Module 2: Regularization

- ✓ Dropout
- ✓ Batch Normalization
- ✓ L1 / L2 Regularization

Module 3: Tuning & Transfer

- ✓ Hyperparameter Tuning
- ✓ Transfer Learning
- ✓ Fine-Tuning Strategies

Module 4: Generalization

- ✓ Bias–Variance Tradeoff
- ✓ Overfitting Strategies
- ✓ Small Dataset Techniques
- ✓ Data Augmentation

Optimization



Optimization

Making training fast, stable, and effective

Why Optimization Matters

- Neural networks learn by minimizing a **loss function** $\mathcal{L}(\theta)$ *→ weights & biases →*
- Challenge:** High-dimensional, non-convex loss landscapes
- Vanilla stochastic gradient descent (SGD) is **slow and unstable**

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

(↺ ↻)

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \text{:- Vanilla SGD}$$

↺ ↻

$$w_{t+1} = w_t - \eta$$

↺ ↻

update *↗ gradient.*

η :- LR

Why Optimization Matters

- Neural networks learn by minimizing a **loss function** $\mathcal{L}(\theta)$
- **Challenge:** High-dimensional, non-convex loss landscapes
- Vanilla stochastic gradient descent (SGD) is **slow and unstable**

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \text{:- Vanilla SGD}$$

update

gradients

- Modern optimizers address:

- Vanishing/exploding gradients ✓
- Saddle points and plateaus
- Noisy gradient estimates
- Ill-conditioned curvature ✓

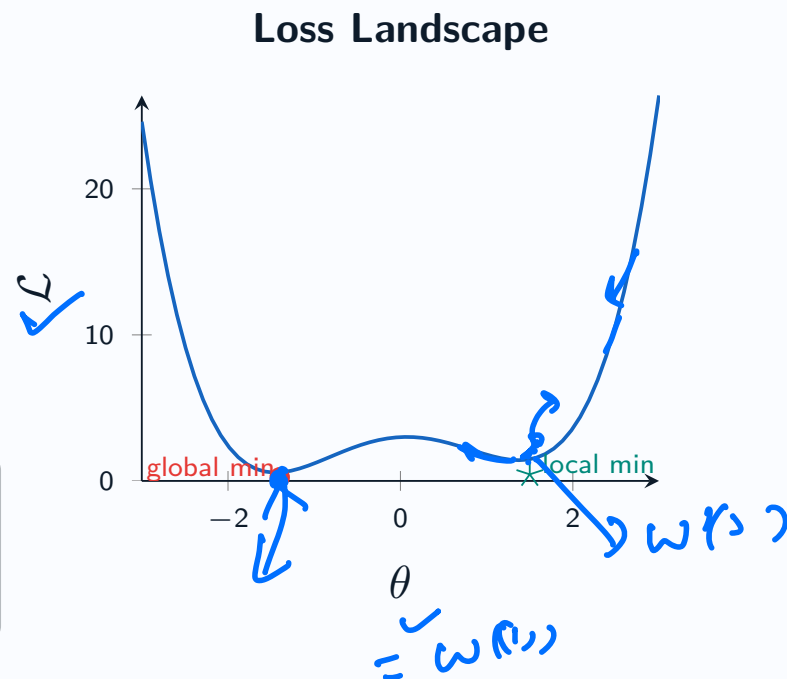
↑ big values
↓ very small
 $w_{t+1} \approx w_t$

Why Optimization Matters

- Neural networks learn by minimizing a **loss function** $\mathcal{L}(\theta)$
- **Challenge:** High-dimensional, non-convex loss landscapes
- Vanilla stochastic gradient descent (SGD) is **slow and unstable**

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \text{:- Vanilla SGD update}$$

- Modern optimizers address:
 - Vanishing/exploding gradients
 - Saddle points and plateaus
 - Noisy gradient estimates
 - Ill-conditioned curvature



SGD Variants

Batch GD

- › Uses **all** training samples $\Rightarrow n=1000$ all
- › Stable but very slow ✓
- › Not practical for large data ✓

$$\nabla \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \nabla \ell_i$$

Batch

$\Rightarrow m=1000 \Rightarrow$ training data

↓
20 at a time $\Rightarrow$

↓
batch size =

↓
100 $\Rightarrow$ 1m training data point

SGD Variants

Batch GD

- › Uses **all** training samples
- › Stable but very slow
- › Not practical for large data

$$\nabla \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \nabla \ell_i$$

Mini-Batch SGD 👍

- › Uses a **mini-batch** of size B
- › Balance of speed & stability
- › Industry standard

$$\nabla \mathcal{L} \approx \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i$$

$B < m$

SGD Variants

Batch GD

- › Uses **all** training samples
- › Stable but very slow
- › Not practical for large data

$$\nabla \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \nabla \ell_i$$

$B=m$

Batch GD
↳ mini Batch SGD
Stochastic GD $\rightarrow B=1$

$\Rightarrow$ iterations

1 | 2
100 | 200
1000 | 2000

$B=64$

Mini-Batch SGD

- › Uses a **mini-batch** of size B
- › Balance of speed & stability
- › Industry standard

$$\nabla \mathcal{L} \approx \frac{1}{B} \sum_{i \in B} \nabla \ell_i$$

B

epochs
1 | 2
1 | 2
1 | 2

$B=1$

Stochastic GD

- › Uses one sample at a time
- › Very noisy updates
- › Can escape local minima

$$\nabla \mathcal{L} \approx \nabla \ell_i$$

$\Rightarrow m=1000$
 $\rightarrow B=1 \rightarrow$
 e

SGD Variants

Batch GD

- › Uses **all** training samples
- › Stable but very slow
- › Not practical for large data

$$\nabla \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \nabla \ell_i$$

Mini-Batch SGD 👍

- › Uses a **mini-batch** of size B
- › Balance of speed & stability
- › Industry standard

$$\nabla \mathcal{L} \approx \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i$$

Stochastic GD

- › Uses **one** sample at a time
- › Very noisy updates
- › Can escape local minima

$$\nabla \mathcal{L} \approx \nabla \ell_i$$

$m=1000 \rightarrow B=100 \Rightarrow 10, 100, 100, \dots, 100$
10 + 100 + 100 + ... + 100

💡 Best Practice

Typical batch sizes: 32, 64, 128, 256. Larger batches $\Rightarrow$ more stable gradients but need tuned LR and more memory.

Momentum Optimizer

Problem with plain SGD:

- Oscillates in ravine-shaped landscapes
- Slow convergence along shallow directions

Momentum Optimizer

Problem with plain SGD:

- Oscillates in ravine-shaped landscapes
- Slow convergence along shallow directions

Momentum adds a velocity term:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} \mathcal{L}$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Momentum Optimizer

Problem with plain SGD:

- Oscillates in ravine-shaped landscapes
- Slow convergence along shallow directions

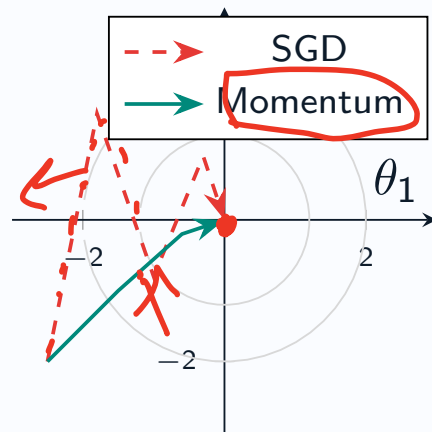
Momentum adds a velocity term:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\theta} \mathcal{L}$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t \rightarrow \text{gradient}$$

- $\beta \approx 0.9$ is the momentum coefficient
- Accumulates gradients like a ball rolling downhill

Momentum vs SGD Path



Key Idea: Adapt the learning rate *per parameter* by scaling it with the **root mean square of past gradients**.

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t$$

RMSProp

Key Idea: Adapt the learning rate *per parameter* by scaling it with the **root mean square of past gradients**.

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t$$

Benefits:

- Larger LR for infrequent features
- Smaller LR for frequent features
- Works well with non-stationary objectives
- Typical: $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Comparison: SGD vs RMSProp

	SGD	RMSProp
Adaptive LR	✗	✓
Per-param LR	✗	✓
Hyperparams	1	2

RMSProp

Key Idea: Adapt the learning rate *per parameter* by scaling it with the **root mean square of past gradients**.

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t$$

Benefits:

- Larger LR for infrequent features
- Smaller LR for frequent features
- Works well with non-stationary objectives
- Typical: $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Comparison: SGD vs RMSProp

	SGD	RMSProp
Adaptive LR	✗	✓
Per-param LR	✗	✓
Hyperparams	1	2

⚠ Note

RMSProp lacks theoretical convergence guarantees in general non-convex settings.

Adam Optimizer

Adam = **A**daptive **M**oment Estimation

Combines *Momentum* + *RMSProp*



vanilla

SGD



$$\omega_{t+1} = \omega_t -$$

$$\frac{\nabla L(\omega_t)}{\sqrt{h_1, h_2, \dots, h_T}}$$

Adam Optimizer

Adam = Adaptive Moment Estimation

Combines Momentum + RMSProp

1st moment (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2nd moment (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Adam Optimizer

Adam = **A**daptive **M**oment Estimation

Combines *Momentum* + *RMSProp*

1st moment (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2nd moment (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

💡 Default Hyperparameters

$$\eta = 0.001$$

$$\beta_1 = 0.9 \text{ (momentum decay)}$$

$$\beta_2 = 0.999 \text{ (variance decay)}$$

$$\epsilon = 10^{-8} \text{ (numerical stability)}$$

Adam Optimizer

Adam = **A**daptive **M**oment Estimation

Combines *Momentum* + *RMSProp*

1st moment (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2nd moment (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

💡 Default Hyperparameters

$$\eta = 0.001$$

$$\beta_1 = 0.9 \text{ (momentum decay)}$$

$$\beta_2 = 0.999 \text{ (variance decay)}$$

$$\epsilon = 10^{-8} \text{ (numerical stability)}$$

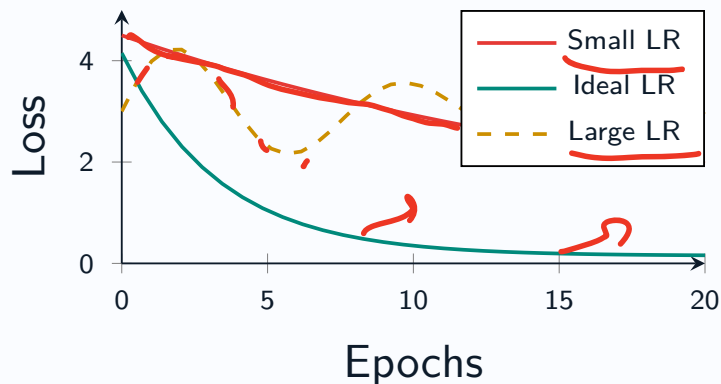
- ✓ Computationally efficient
- ✓ Works with sparse gradients
- ✓ Little memory requirement
- ✓ Invariant to gradient scaling
- ⚠ May generalize worse than SGD+momentum in some tasks

Learning Rate: The Most Critical Hyperparameter

- Controls the **step size** of each update
- Too large** $\Rightarrow$ divergence / oscillation
- Too small** $\Rightarrow$ slow convergence / stuck
- Just right** $\Rightarrow$ fast, stable learning

$$\eta = \text{LR}$$

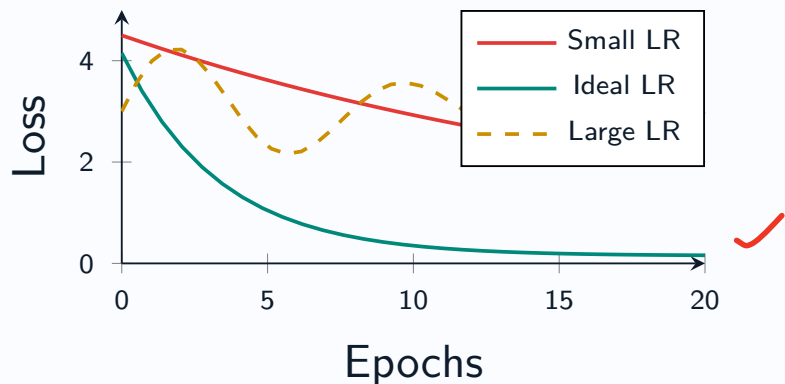
Effect of Learning Rate



Learning Rate: The Most Critical Hyperparameter

- › Controls the **step size** of each update
- › **Too large** $\Rightarrow$ divergence / oscillation
- › **Too small** $\Rightarrow$ slow convergence / stuck
- › **Just right** $\Rightarrow$ fast, stable learning

Effect of Learning Rate



LR Range Test

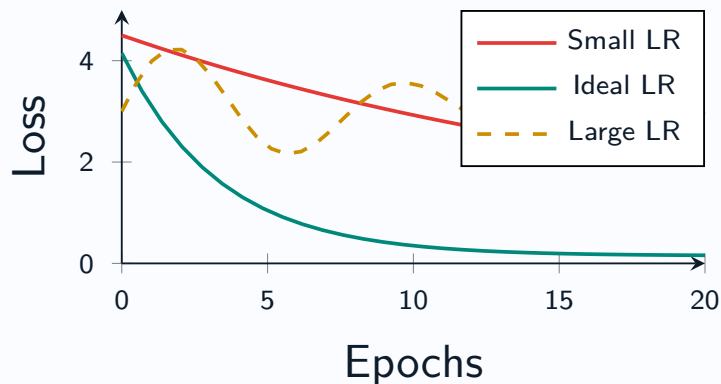
Start with very small LR, increase exponentially each step.

Plot loss vs LR. Choose LR just **before** the loss stops falling.

Learning Rate: The Most Critical Hyperparameter

- › Controls the **step size** of each update
- › **Too large** $\Rightarrow$ divergence / oscillation
- › **Too small** $\Rightarrow$ slow convergence / stuck
- › **Just right** $\Rightarrow$ fast, stable learning

Effect of Learning Rate



LR Range Test

Start with very small LR, increase exponentially each step.

Plot loss vs LR. Choose LR just **before** the loss stops falling.

Rule of thumb:

LR $\approx 10^{-3}$ for Adam

LR $\approx 10^{-2}$ for SGD

Regularization

Regularization

Preventing overfitting and improving generalization

Bias–Variance Tradeoff

$$\mathbb{E}[\text{Error}^2] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

Bias–Variance Tradeoff

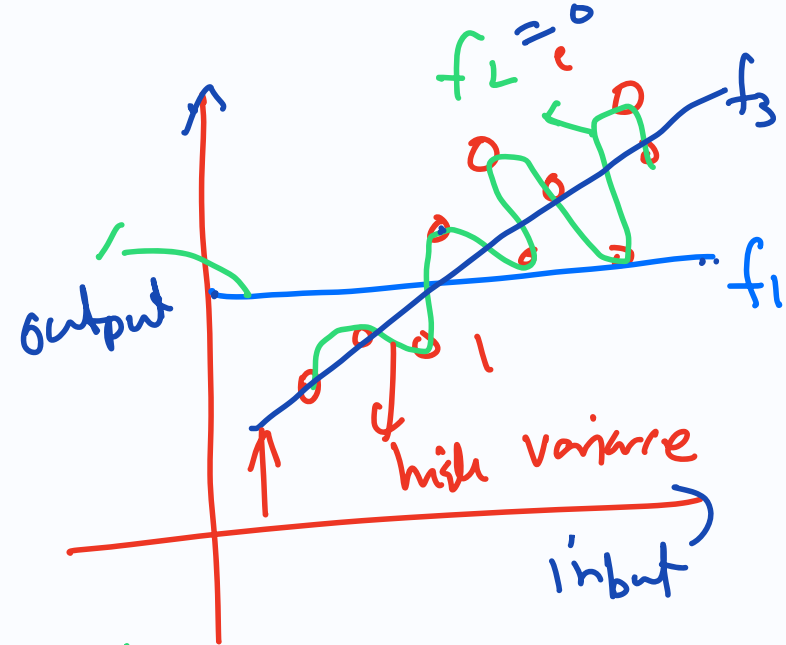
$$\mathbb{E}[\text{Error}^2] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \underbrace{\sigma^2}_{\text{irreducible}} \rightarrow$$

- **High Bias:** Model too simple; misses patterns. $\rightarrow f_1$

○ Add layers, units, features

- **High Variance:** Model memorizes training data f_2

○ Regularize, more data, dropout



	underfitted model	bias	variance
f_1	underfitted	high	0
f_2	overfitting	0	very high
f_3	or best	okay	okay

Bias–Variance Tradeoff

$$\mathbb{E}[\text{Error}^2] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

➤ **High Bias:** Model too simple; misses patterns

○ Add layers, units, features

➤ **High Variance:** Model memorizes training data

○ Regularize, more data, dropout

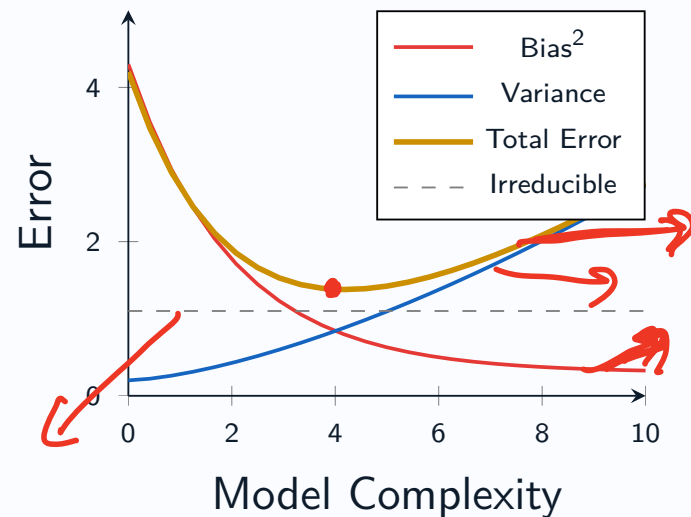
💡 Diagnostic Tool

Compare **train error** vs **val error**.

Large train error $\Rightarrow$ high bias. ✓

Small train, large val $\Rightarrow$ high variance. ✓

Bias-Variance Tradeoff



L1 and L2 Regularization

L2 Regularization (Weight Decay)

Add penalty proportional to $\|\mathbf{w}\|_2^2$:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + 0.5\lambda \sum_j w_j^2$$

- Penalizes large weights
- Encourages **small, distributed** weights
- Smooth shrinkage toward zero

L1 Regularization (Lasso)

Add penalty proportional to $\|\mathbf{w}\|_1$:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \sum_j |w_j|$$

- Induces **sparsity** in weights
- Many weights $\rightarrow$ exactly 0
- Feature selection effect

L1 and L2 Regularization

L2 Regularization (Weight Decay)

Add penalty proportional to $\|\mathbf{w}\|_2^2$:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + 0.5\lambda \sum_j w_j^2$$

- Penalizes large weights
- Encourages **small, distributed** weights
- Smooth shrinkage toward zero

L1 Regularization (Lasso)

Add penalty proportional to $\|\mathbf{w}\|_1$:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \sum_j |w_j|$$

- Induces **sparsity** in weights
- Many weights $\rightarrow$ exactly 0
- Feature selection effect

💡 Elastic Net

Combine both: $\mathcal{L}_{\text{reg}} = \mathcal{L} + \alpha\lambda\|\mathbf{w}\|_1 + \frac{(1-\alpha)\lambda}{2}\|\mathbf{w}\|_2^2$. Best of both worlds.

Dropout Regularization

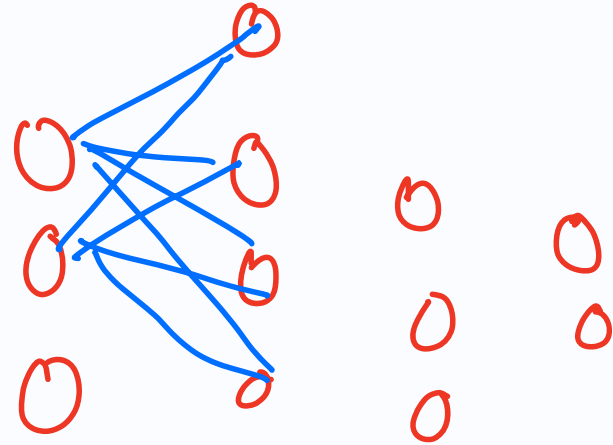
another way of dealing with overfitting

Idea: Randomly **zero out** neurons during training with probability p .

For each unit j at training time:

$$r_j \sim \text{Bernoulli}(1 - p)$$

$$\tilde{h}_j = r_j \cdot h_j$$



Dropout Regularization

Idea: Randomly **zero out** neurons during training with probability p .

For each unit j at training time:

$$r_j \sim \text{Bernoulli}(1 - p)$$

$$\tilde{h}_j = r_j \cdot h_j$$

Why it works:

- Prevents **co-adaptation** of neurons
- Forces redundant representations
- Reduces **internal covariate shift**

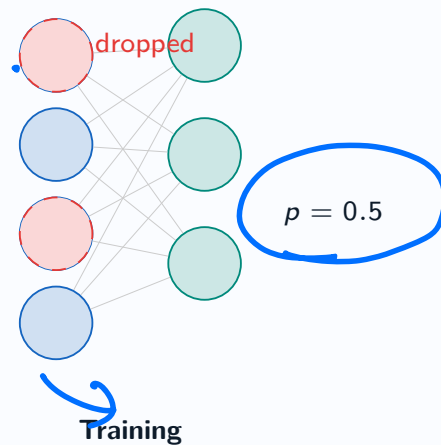
Dropout Regularization

Idea: Randomly **zero out** neurons during training with probability p .

For each unit j at training time:

$$r_j \sim \text{Bernoulli}(1 - p)$$

$$\tilde{h}_j = r_j \cdot h_j$$



Why it works:

- Prevents **co-adaptation** of neurons
- Forces redundant representations
- Reduces **internal covariate shift**

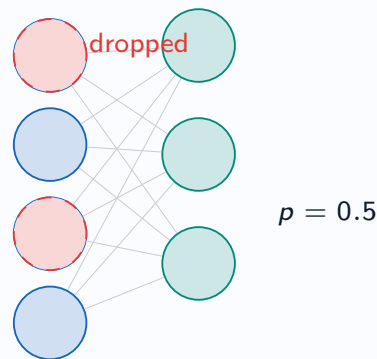
Dropout Regularization

Idea: Randomly **zero out** neurons during training with probability p .

For each unit j at training time:

$$r_j \sim \text{Bernoulli}(1 - p)$$

$$\tilde{h}_j = r_j \cdot h_j$$



Training

Why it works:

- Prevents **co-adaptation** of neurons
- Forces redundant representations
- Reduces **internal covariate shift**

⚠ Typical values

Dense layers: $p = 0.5$

Conv layers: $p = 0.1-0.3$

Input layer: $p = 0.2$

Batch Normalization

Problem: Internal covariate shift —
distribution of layer inputs changes during
training, slowing convergence.

Batch Normalization

Problem: Internal covariate shift — distribution of layer inputs changes during training, slowing convergence.

Solution: Normalize activations within each mini-batch.

Batch Normalization

Problem: Internal covariate shift — distribution of layer inputs changes during training, slowing convergence.

↓ CNN
→ responsibility

Solution: Normalize activations within each mini-batch.

For each feature k in a mini-batch $\mathcal{B}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_i x_i^{(k)}$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_i (x_i^{(k)} - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)}$$

Batch Normalization

Problem: Internal covariate shift — distribution of layer inputs changes during training, slowing convergence.

Solution: Normalize activations within each mini-batch.

Learnable parameters: γ , β (scale and shift)

For each feature k in a mini-batch $\mathcal{B}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_i x_i^{(k)}$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_i (x_i^{(k)} - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)}$$

Batch Normalization

Problem: Internal covariate shift — distribution of layer inputs changes during training, slowing convergence.

Solution: Normalize activations within each mini-batch.

For each feature k in a mini-batch $\mathcal{B}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_i x_i^{(k)}$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_i (x_i^{(k)} - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)}$$

Learnable parameters: γ, β (scale and shift)

💡 Benefits

- Allows higher learning rates
- Less sensitive to weight init
- Acts as regularizer (may drop dropout)
- Faster convergence

Batch Normalization

Problem: Internal covariate shift — distribution of layer inputs changes during training, slowing convergence.

Solution: Normalize activations within each mini-batch.

For each feature k in a mini-batch $\mathcal{B}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_i x_i^{(k)}$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_i (x_i^{(k)} - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)}$$

Learnable parameters: γ, β (scale and shift)

💡 Benefits

- Allows higher learning rates
- Less sensitive to weight init
- Acts as regularizer (may drop dropout)
- Faster convergence

⚠ Placement

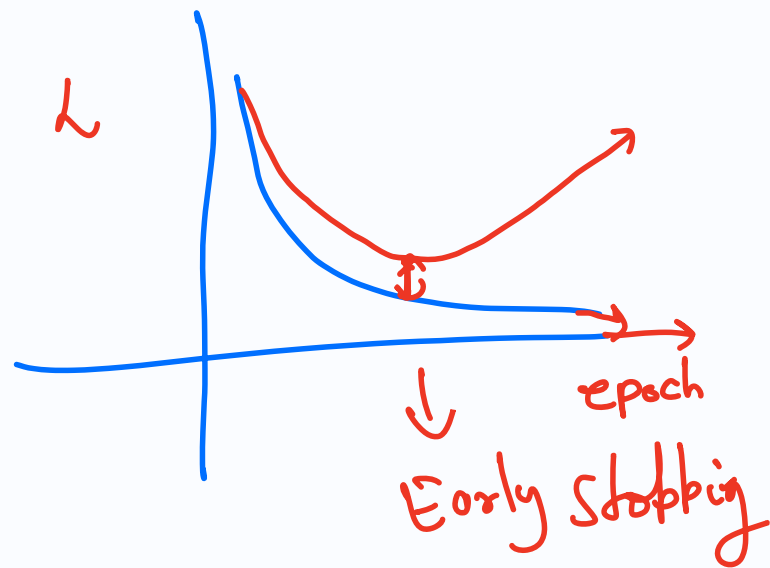
Place BN **after** the linear transformation and **before** the activation. Common: Conv $\rightarrow$ BN $\rightarrow$ ReLU.

Early Stopping

↓ way to avoid overfitting.

Concept: Monitor validation loss and stop training when it stops improving.

- Simple and effective regularization
- No additional computation needed
- Save the **best checkpoint**



Early Stopping

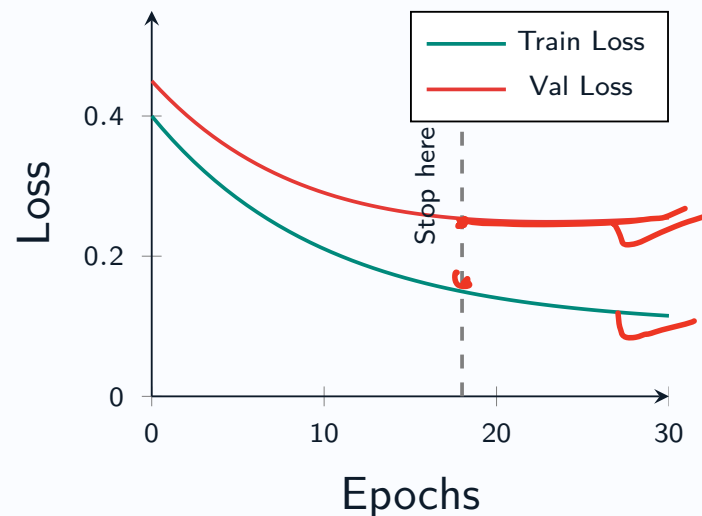
Concept: Monitor validation loss and stop training when it stops improving.

- Simple and effective regularization
- No additional computation needed
- Save the **best checkpoint**

⚠ Pitfall

Don't use the test set for early stopping. Always use a separate **validation set**.

Early Stopping



Idea: Artificially expand training set with label-preserving transformations.

Data Augmentation

Idea: Artificially expand training set with label-preserving transformations.

Image Augmentations:

- Random flip, crop, rotation → x_1
- Color jitter, brightness/contrast → x_1
- Cutout / Random Erasing
- Mixup: $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$
- CutMix: paste patches between images

Hyperparameter Tuning



Hyperparameter Tuning

Finding the best configuration for your model

What Are Hyperparameters?

Model Hyperparameters:

- Number of layers & units per layer *→ neurons*
- Activation functions
- Architecture choices (skip connections, etc.)

What Are Hyperparameters?

Model Hyperparameters:

- › Number of layers & units per layer
- › Activation functions
- › Architecture choices (skip connections, etc.)

Optimization Hyperparameters:

- › Learning rate η
- › Batch size B
- › Optimizer choice & its parameters
- › Number of epochs

What Are Hyperparameters?

Model Hyperparameters:

- › Number of layers & units per layer
- › Activation functions
- › Architecture choices (skip connections, etc.)

Optimization Hyperparameters:

- › Learning rate η
- › Batch size B
- › Optimizer choice & its parameters
- › Number of epochs

Regularization Hyperparameters:

- › Dropout rate p 0.5
- › Weight decay λ
- › L1/L2 penalty strength

What Are Hyperparameters?

Model Hyperparameters:

- › Number of layers & units per layer
- › Activation functions
- › Architecture choices (skip connections, etc.)

Optimization Hyperparameters:

- › Learning rate η
- › Batch size B
- › Optimizer choice & its parameters
- › Number of epochs

Regularization Hyperparameters:

- › Dropout rate p
- › Weight decay λ
- › L1/L2 penalty strength

⚠ Key Principle

Hyperparameters are **not learned** during training. They must be set before training and validated on a held-out validation set.

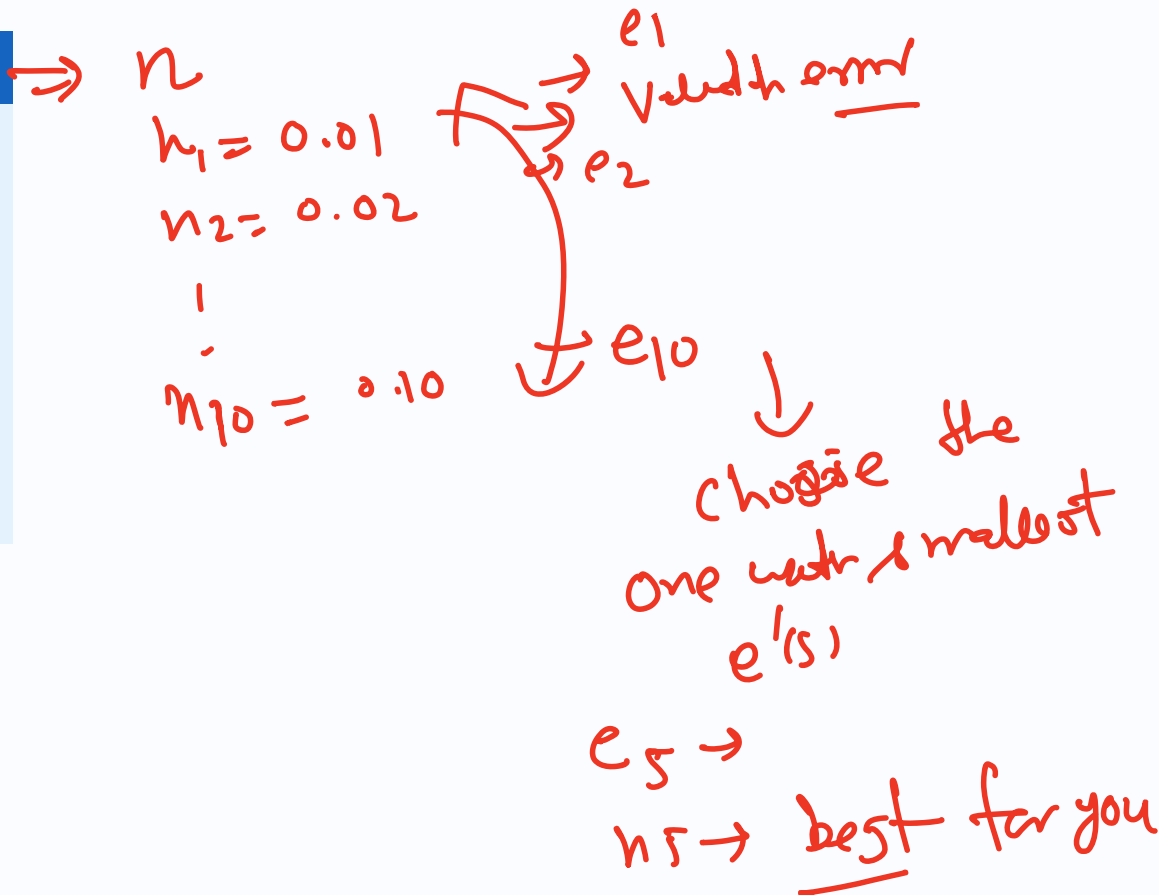
Grid Search vs Random Search

Grid Search

- › Exhaustively try all combinations
- › Guaranteed to find best in grid
- › Scales **exponentially** with # params
- › Good for ≤ 2 hyperparameters

●	●	●	●
●	●	●	●
●	●	●	●
●	●	●	●

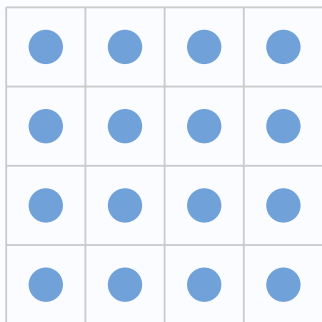
Grid Search (16 evals)



Grid Search vs Random Search

Grid Search

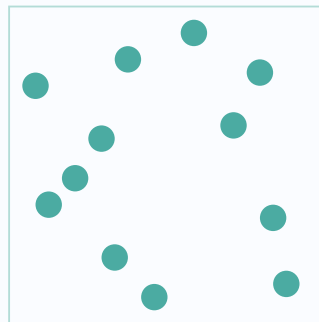
- › Exhaustively try all combinations
- › Guaranteed to find best in grid
- › Scales **exponentially** with # params
- › Good for ≤ 2 hyperparameters



Grid Search (16 evals)

Random Search 👍

- › Randomly sample from distributions
- › More efficient than grid (Bergstra & Bengio 2012)
- › Explores **more unique values** per param
- › Good for > 3 hyperparameters



Random Search (12 evals)

K-Fold Cross-Validation:

- Split data into K folds
- Train on $K - 1$, validate on 1
- Average metrics across all folds
- More reliable than single split

K-Fold Cross-Validation:

- Split data into K folds
- Train on $K - 1$, validate on 1
- Average metrics across all folds
- More reliable than single split

Train / Val / Test Split:

Typical: 70% train / 15% val / 15% test

For large datasets: 98% / 1% / 1%

Cross-Validation & Model Selection

K-Fold Cross-Validation:

- Split data into K folds
- Train on $K - 1$, validate on 1
- Average metrics across all folds
- More reliable than single split

Fold 1	Validation	Train	Train	Train	Train
Fold 2	Train	Validation	Train	Train	Train
Fold 3	Train	Train	Validation	Train	Train
Fold 4	Train	Train	Train	Validation	Train
Fold 5	Train	Train	Train	Train	Validation

Validation Train

Train / Val / Test Split:

Typical: 70% train / 15% val / 15% test

For large datasets: 98% / 1% / 1%

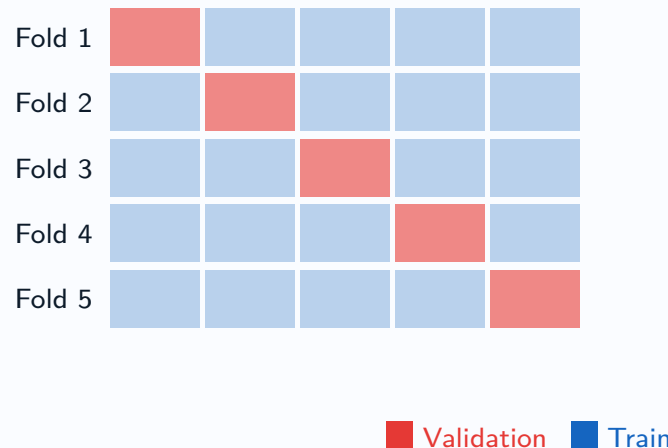
Cross-Validation & Model Selection

K-Fold Cross-Validation:

- Split data into K folds
- Train on $K - 1$, validate on 1
- Average metrics across all folds
- More reliable than single split

Train / Val / Test Split:

Typical: 70% train / 15% val / 15% test
For large datasets: 98% / 1% / 1%



⚠ Data Leakage ✓

Never use test set during model development or hyperparameter selection!

Weight Initialization

$$\underline{\omega_0} \Rightarrow$$

$$\omega_{t+1} = \omega_t - n \cdot \Delta L_{\omega_t}$$

Why it matters:

- Bad init $\Rightarrow$ vanishing / exploding gradients
- Symmetry breaking is essential

Transfer Learning

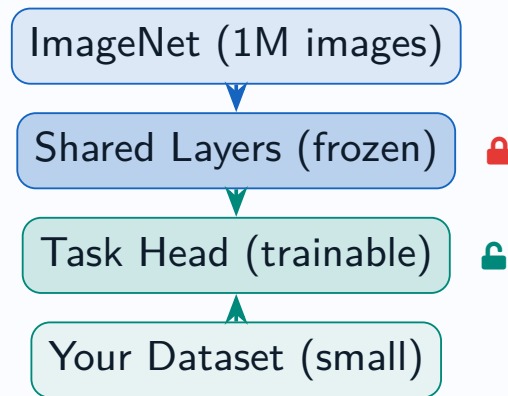
Transfer Learning

Leveraging pretrained models for new tasks

Transfer Learning: Concept

Motivation:

- Training large models from scratch is expensive
- Many tasks share common low-level features
- Pretrained models encode rich representations



Strategy:

1. Take a model pretrained on large dataset
2. Freeze the early layers (feature extractor)

Popular Pretrained Models

Computer Vision

- › **ResNet-50/101/152** — residual networks
- › **EfficientNet B0-B7** — accuracy/efficiency
- › **ViT** — Vision Transformer
- › **CLIP** — vision-language alignment
- › **DINOv2** — self-supervised ViT

Popular Pretrained Models

Computer Vision ✓

- **ResNet-50/101/152** — residual networks
- **EfficientNet B0-B7** — accuracy/efficiency
- **ViT** — Vision Transformer
- **CLIP** — vision-language alignment
- **DINOv2** — self-supervised ViT

A NLP ✓ Text

- **BERT** / **RoBERTa** — bidirectional encoders
- **GPT-2/3/4** — autoregressive LMs
- **T5** / **BART** — seq2seq models
- **Sentence-BERT** — sentence embeddings
- **LLaMA 3** — open-source LLM

Popular Pretrained Models

Computer Vision

- **ResNet-50/101/152** — residual networks
- **EfficientNet B0-B7** — accuracy/efficiency
- **ViT** — Vision Transformer
- **CLIP** — vision-language alignment
- **DINOv2** — self-supervised ViT

A NLP / Text

- **BERT / RoBERTa** — bidirectional encoders
- **GPT-2/3/4** — autoregressive LMs
- **T5 / BART** — seq2seq models
- **Sentence-BERT** — sentence embeddings
- **LLaMA 3** — open-source LLM

Where to find them

-  huggingface.co/models — 500,000+ pretrained models
-  pytorch.org/hub — PyTorch Hub ✓
-  tfhub.dev — TensorFlow Hub ✓

Overfitting and Small Datasets



Overfitting and Small Datasets

Generalization strategies when data is scarce

Diagnosing Overfitting

Signs of Overfitting:

- Training loss $\ll$ Validation loss
- High training accuracy, low val accuracy
- Val loss starts increasing after some point
- Model memorizes training examples

Diagnosing Overfitting

Signs of Overfitting:

- Training loss $\ll$ Validation loss
- High training accuracy, low val accuracy
- Val loss starts increasing after some point
- Model memorizes training examples

Root Causes:

- Model capacity too high for data
- Insufficient training data
- Too many training epochs
- Too little regularization

Diagnosing Overfitting

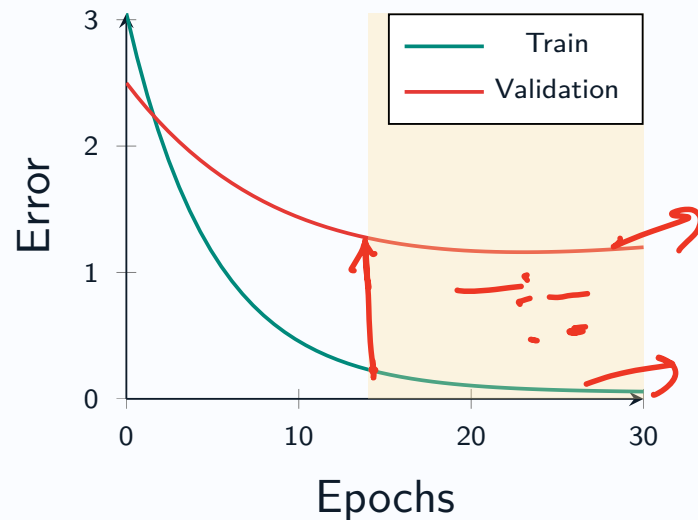
Signs of Overfitting:

- Training loss $\ll$ Validation loss
- High training accuracy, low val accuracy
- Val loss starts increasing after some point
- Model memorizes training examples

Root Causes:

- Model capacity too high for data
- Insufficient training data
- Too many training epochs
- Too little regularization

Overfitting Signature



Strategies to Combat Overfitting

More Data ✓

- Collect more labeled samples ✓
- Data augmentation ✓
- Synthetic data generation ✓
- Semi-supervised learning ✓

Strategies to Combat Overfitting

More Data

- Collect more labeled samples
- Data augmentation
- Synthetic data generation
- Semi-supervised learning

Reduce Model Complexity ✓

- Fewer layers or units
- Simpler architecture
- Feature selection ✓

Strategies to Combat Overfitting

More Data

- Collect more labeled samples
- Data augmentation
- Synthetic data generation
- Semi-supervised learning

Regularization

- Dropout ✓
- L1/L2 penalty ✓
- Weight decay ✓
- Noise injection ✓

Reduce Model Complexity

- Fewer layers or units
- Simpler architecture
- Feature selection

Strategies to Combat Overfitting

More Data

- Collect more labeled samples
- Data augmentation
- Synthetic data generation
- Semi-supervised learning

Regularization

- Dropout
- L1/L2 penalty
- Weight decay
- Noise injection

Reduce Model Complexity

- Fewer layers or units
- Simpler architecture
- Feature selection

Training Strategies ✓

- Early stopping ✓
- Learning rate warmup ✓
- Stochastic depth
- Label smoothing

Practical Deep Learning Checklist

Before Training

- ✓ Normalize input data ✓
- ✓ Use appropriate weight init ✓
- ✓ Set up validation split ✓
- ✓ Choose architecture & optimizer ✓
- ✓ Check data pipeline is correct ✓

Practical Deep Learning Checklist

Before Training

- ✓ Normalize input data
- ✓ Use appropriate weight init
- ✓ Set up validation split
- ✓ Choose architecture & optimizer
- ✓ Check data pipeline is correct

During Training

- ✓ Monitor train vs val loss ✓
- ✓ Use learning rate scheduler ✓
- ✓ Apply early stopping ✓
- ✓ Log metrics (W&B, TensorBoard)
- ✓ Save best checkpoint

Practical Deep Learning Checklist

Before Training

- ✓ Normalize input data
- ✓ Use appropriate weight init
- ✓ Set up validation split
- ✓ Choose architecture & optimizer
- ✓ Check data pipeline is correct

After Training

- ✓ Evaluate on held-out test set ✓
- ✓ Analyze confusion/matrix ^{MSE}
- ✓ Check for data leakage ✓
- ✓ Profile inference speed ✓
- ✓ Quantize/prune if needed }

During Training

- ✓ Monitor train vs val loss
- ✓ Use learning rate scheduler
- ✓ Apply early stopping
- ✓ Log metrics (W&B, TensorBoard)
- ✓ Save best checkpoint

Practical Deep Learning Checklist

Before Training

- ✓ Normalize input data
- ✓ Use appropriate weight init
- ✓ Set up validation split
- ✓ Choose architecture & optimizer
- ✓ Check data pipeline is correct

After Training

- ✓ Evaluate on held-out test set
- ✓ Analyze confusion matrix
- ✓ Check for data leakage
- ✓ Profile inference speed
- ✓ Quantize/prune if needed

During Training

- ✓ Monitor train vs val loss
- ✓ Use learning rate scheduler
- ✓ Apply early stopping
- ✓ Log metrics (W&B, TensorBoard)
- ✓ Save best checkpoint

💡 Golden Rule

Build a simple baseline first. Overfit intentionally on a tiny batch. Then scale up and regularize.

Common Mistakes & How to Fix Them

✗ Mistake 1: Wrong LR

Loss explodes or doesn't decrease.

Fix: Use LR range test; start with 10^{-3}
for Adam.

Common Mistakes & How to Fix Them

✖ Mistake 1: Wrong LR

Loss explodes or doesn't decrease.

Fix: Use LR range test; start with 10^{-3} for Adam.

✖ Mistake 2: No Normalization

Slow convergence, vanishing gradients.

Fix: Normalize inputs; add BatchNorm after linear layers.

Common Mistakes & How to Fix Them

✖ Mistake 1: Wrong LR

Loss explodes or doesn't decrease.

Fix: Use LR range test; start with 10^{-3} for Adam.

✖ Mistake 2: No Normalization

Slow convergence, vanishing gradients.

Fix: Normalize inputs; add BatchNorm after linear layers.

✖ Mistake 3: Data Leakage

Suspiciously high test accuracy.

Fix: Split data *before* any preprocessing.
Never look at test set.

✖ Mistake 4: Ignoring Overfitting

High train, low val accuracy.

Fix: Add dropout, L2 regularization,
data augmentation. ✓

Common Mistakes & How to Fix Them

✖ Mistake 1: Wrong LR

Loss explodes or doesn't decrease.

Fix: Use LR range test; start with 10^{-3} for Adam.

✖ Mistake 2: No Normalization

Slow convergence, vanishing gradients.

Fix: Normalize inputs; add BatchNorm after linear layers.

✖ Mistake 3: Data Leakage

Suspiciously high test accuracy.

Fix: Split data *before* any preprocessing.
Never look at test set.

✖ Mistake 4: Ignoring Overfitting

High train, low val accuracy.

Fix: Add dropout, L2 regularization, data augmentation.

✖ Mistake 5: Poor Init

Gradients stuck at zero, symmetry.

Fix: Use He init with ReLU, Xavier with Tanh.

Common Mistakes & How to Fix Them

✘ Mistake 1: Wrong LR

Loss explodes or doesn't decrease.

Fix: Use LR range test; start with 10^{-3} for Adam.

✘ Mistake 2: No Normalization

Slow convergence, vanishing gradients.

Fix: Normalize inputs; add BatchNorm after linear layers.

✘ Mistake 3: Data Leakage

Suspiciously high test accuracy.

Fix: Split data *before* any preprocessing. Never look at test set.

✘ Mistake 4: Ignoring Overfitting

High train, low val accuracy.

Fix: Add dropout, L2 regularization, data augmentation.

✘ Mistake 5: Poor Init

Gradients stuck at zero, symmetry.

Fix: Use He init with ReLU, Xavier with Tanh.

✘ Mistake 6: No Validation Monitoring

Model overtrained without knowing.

Fix: Always use early stopping with patience.

Summary: Techniques at a Glance

Technique	Purpose	When to Use	Key Param
Adam	Adaptive optimization	Default starting point	$\eta = 10^{-3}$
Dropout	Reduce overfitting	Dense layers	$p = 0.5$
BatchNorm	Stabilize training	After linear/conv	After layer
L2 / Weight Decay	Reduce overfitting	Any model	$\lambda = 10^{-4}$
Early Stopping	Prevent overtraining	All training	patience
Transfer Learning	Scarce data	Any domain	LR ratio
Data Augm.	Expand training set	Image/audio/text	aug policy
Label Smoothing	Regularization	Classification	$\epsilon = 0.1$